



1. Sender Program (sender.c)

```
[madhumitha@localhost ~]$ cat sender.c
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#define SharedMemSize 50
int main()
{
    int shmid;
    key_t key;
    char *shared_memory;
    key = 5677;

    if ((shmid = shmget(key, SharedMemSize, IPC_CREAT | 0666)) < 0) {
        perror("shmget");
        exit(1);
    }
    if((shared_memory = shmat(shmid, NULL, 0)) == (char *) -1) {
        perror("shmat");
        exit(1);
    }
    sprintf(shared_memory, "Welcome to Shared Memory from Sender!");
    printf("Message sent to shared memory.\n");
    sleep(2);
    return 0;
}
```



2. Receiver Program (receiver.c)

```
[madhumitha@localhost ~]$ cat receiver.c
```

```
#include <sys/types.h>
```

```
#include <sys/ipc.h>
```

```
#include <sys/shm.h>
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define SharedMemSize 50
```

```
int main()
```

```
{
```

```
    int shmid;
```

```
    key_t key;
```

```
    char *shared_memory;
```

```
    key = 5677;
```

```
    if ((shmid = shmget(key, SharedMemSize, 0666)) < 0) {
```

```
        perror("shmget");
```

```
        exit(1);
```

```
    }
```

```
    if((shared_memory = shmat(shmid, NULL, 0)) == (char *) -1) {
```

```
        perror("shmat");
```

```
        exit(1);
```

```
    }
```

```
    printf("Message Received: %s\n", shared_memory);
```

```
    return 0;
```

```
}
```



madhumitha@localhost:~



3. Compile the Programs

```
[madhumitha@localhost ~]$ gcc sender.c -o sender
```

```
[madhumitha@localhost ~]$ gcc receiver.c -o receiver
```

```
[madhumitha@localhost ~]$ ls -l sender receiver
```

```
-rwxrwxr-x 1 madhumitha madhumitha 16832 May 24 13:10 receiver
-rwxrwxr-x 1 madhumitha madhumitha 17008 May 24 13:10 sender
```

```
[madhumitha@localhost ~]$
```

4. Run the Programs

```
[madhumitha@localhost ~]$ ./receiver
```

```
Message Received: Welcome to Shared Memory from Sender!
```

```
[madhumitha@localhost ~]$ ./sender
```

```
Message sent to shared memory.
```

```
[madhumitha@localhost ~]$ █
```



madhumitha@localhost:~



5. Execution (Step by Step)

Step 1: Open two terminals.

Step 2: Run the receiver program first (it waits for data).

```
[madhumitha@localhost ~]$ ./receiver
```

```
Message Received: Welcome to Shared Memory from Sender!
```

Step 3: Run the sender program in another terminal.

```
[madhumitha@localhost ~]$ ./sender
```

```
Message sent to shared memory.
```

Step 4: The receiver reads the message written by the sender from the shared memory.

```
[madhumitha@localhost ~]$ █
```